BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

AIMLCZG546 — Software Engineering for Machine Learning

ASSIGNMENT I

Requirements Formulation & System Architecture for Automated Loan Underwriting

Group No: 84

Sl	BITS ID	Name	Qualitative Contribution	%
1	2025AA05710	Singh Pritesh Sanjay Poonam	Quality-Attribute Testing, Performance Benchmarking, System Integration	25%
2	2025AA05368	Gangera Tushar Kantibhai Dayaben	Requirements Formulation, Problem Statement, GR4ML Modeling	25%
3	2025AB05154	Gangam Shuba Nandini	Feature Engineering, ML Pipeline Design, Model Training	25%
4	2025AA05574	Shaifali Garg	System Architecture, FastAPI Serving, Event Simulation	25%

Weightage: 10 marks | Deliverables: report (this PDF), executable notebook (Group_84.ipynb), and source repository.

Objective 1 — Requirements Formulation

1. Domain & Problem Statement

Domain: Consumer Credit Risk Assessment & Automated Loan Underwriting.

Problem Statement: Traditional manual underwriting of consumer credit is slow, labour-intensive and susceptible to cognitive bias. This project implements a real-time, Machine-Learning-based decision-support service that automates loan underwriting. The service consumes applicant demographics, credit metrics and asset–liability levels and classifies each application as **Approved** or **Denied** with a risk tier (LOW / MEDIUM / HIGH). To protect the downstream estimator and guarantee a premium user experience, the system enforces a strict Pydantic validation boundary (100% of malformed payloads rejected) and keeps average inference latency well under a 150 ms SLA.

2. GR4ML Requirement Specifications & Goals

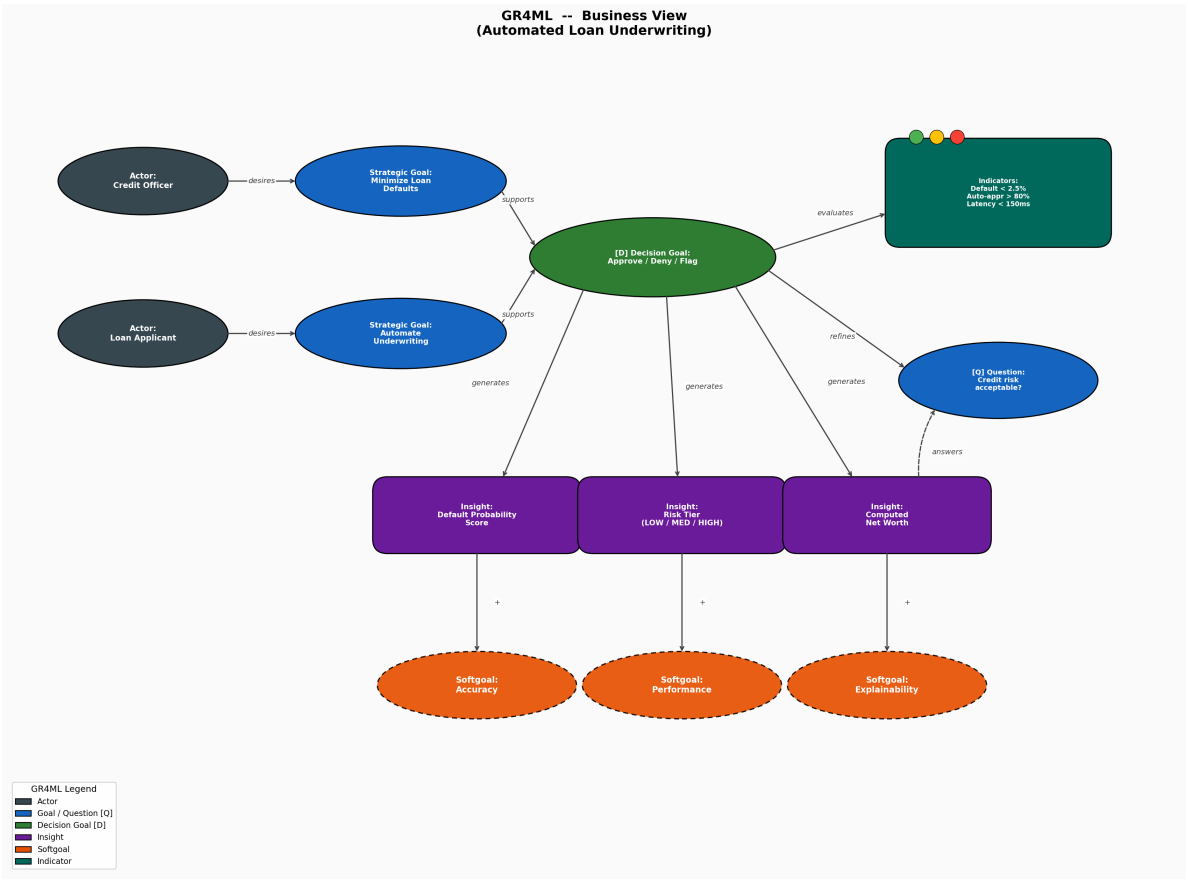
I. Business View

Actors	Loan Applicant (submits financial details); Credit Officer (reviews flagged applications)
Strategic Goals	Minimise loan defaults (risk control); automate application flow (operational efficiency)
Decision Goal	Approve / Deny / Flag the loan application
Question Goal	Is the applicant's credit risk within acceptable limits?
Indicators	Default rate < 2.5%; auto-approval rate > 80%; response latency < 150 ms
Insights	Default-probability score; risk tier (LOW/MED/HIGH); computed net worth

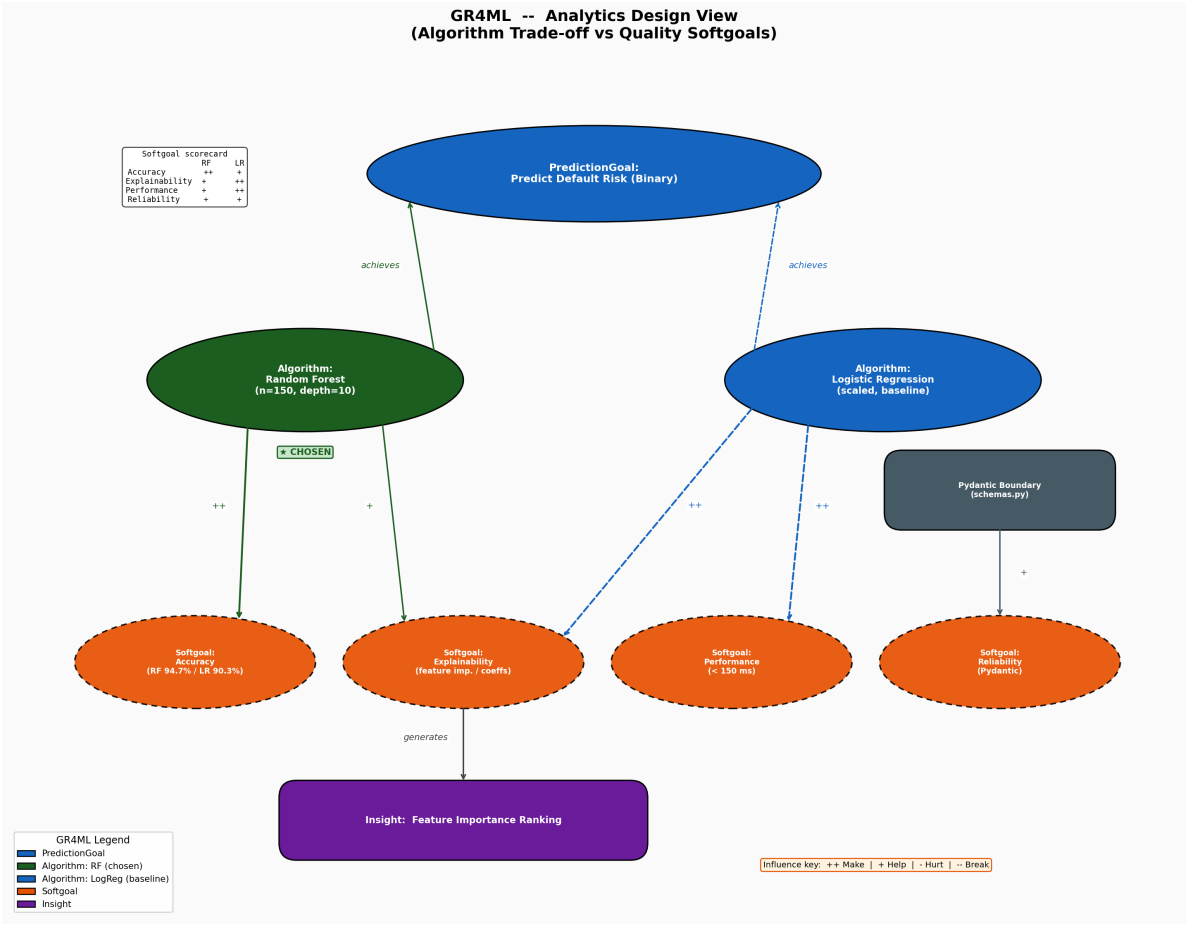
II. Analytics Design & III. Data Preparation Views

Analytics Goal	Prediction — binary classification of default risk
Candidate Algorithms	Random Forest (chosen) vs Logistic Regression (explainable baseline)
Softgoals	Accuracy, Performance (latency), Explainability (feature importance), Reliability (Pydantic)
Raw Entities	Loan applications (demographics), credit-bureau history, asset registry
Prep Tasks	Drop non-predictive / redundant / low-importance columns; categorical encoding; feature engineering
Operators	Correlation filter, importance filter, LabelEncoder, ratio engineering, Pydantic validators

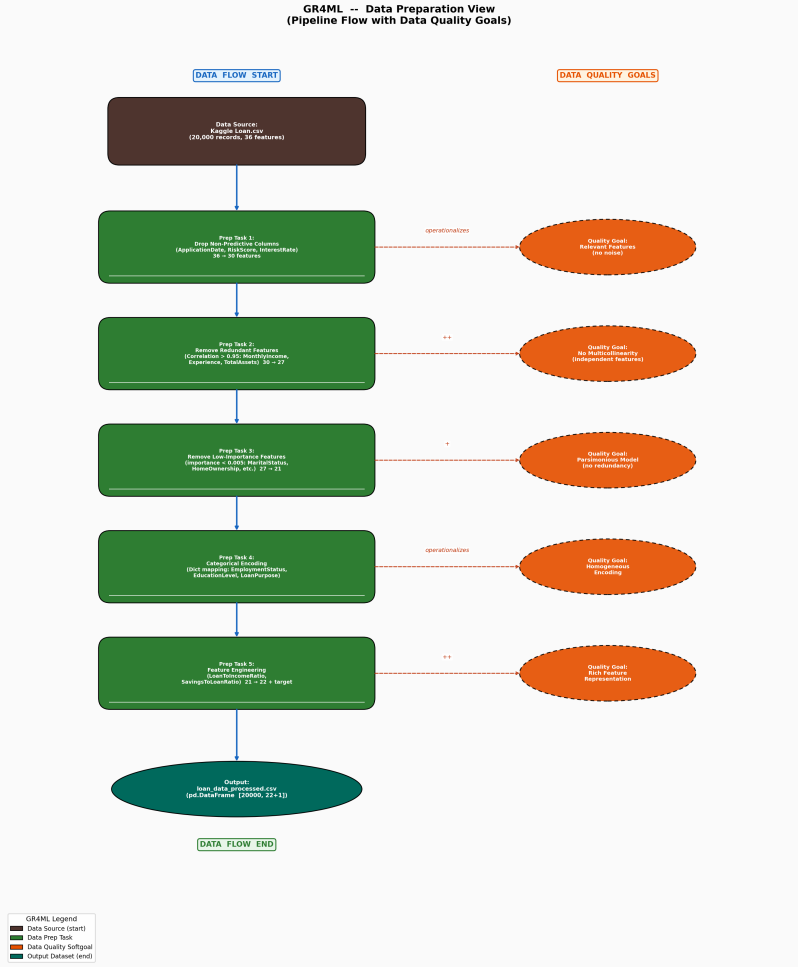
3. GR4ML Graphical Notations



GR4ML Business View — actors, strategic/decision/question goals, indicators, insights and softgoals.



GR4ML Analytics Design View — two candidate algorithms scored against the quality softgoals; Random Forest wins the dominant Accuracy softgoal and is selected.



GR4ML Data Preparation View — five-stage pipeline (36→22 features) and the data-quality goal each stage operationalises.

4. Top Three Quality Requirements

1. Robustness (Pydantic boundary validation). ML estimators cannot natively handle out-of-range or corrupted inputs. A strict validation filter (credit score $\in$ [300, 850], age $\geq$ 18, loan $\leq$ 500% of income) stops bad inputs from causing silent failures inside the model. *Metric:* 100% of invalid inputs rejected at the API boundary (HTTP 422/400).

2. Reliability (type-safe, well-formed outputs). Every response conforms to the `LoanApprovalResult` JSON schema, enabling seamless integration with downstream ledgers. *Metric:* zero unhandled 500 responses; deterministic risk-tier mapping.

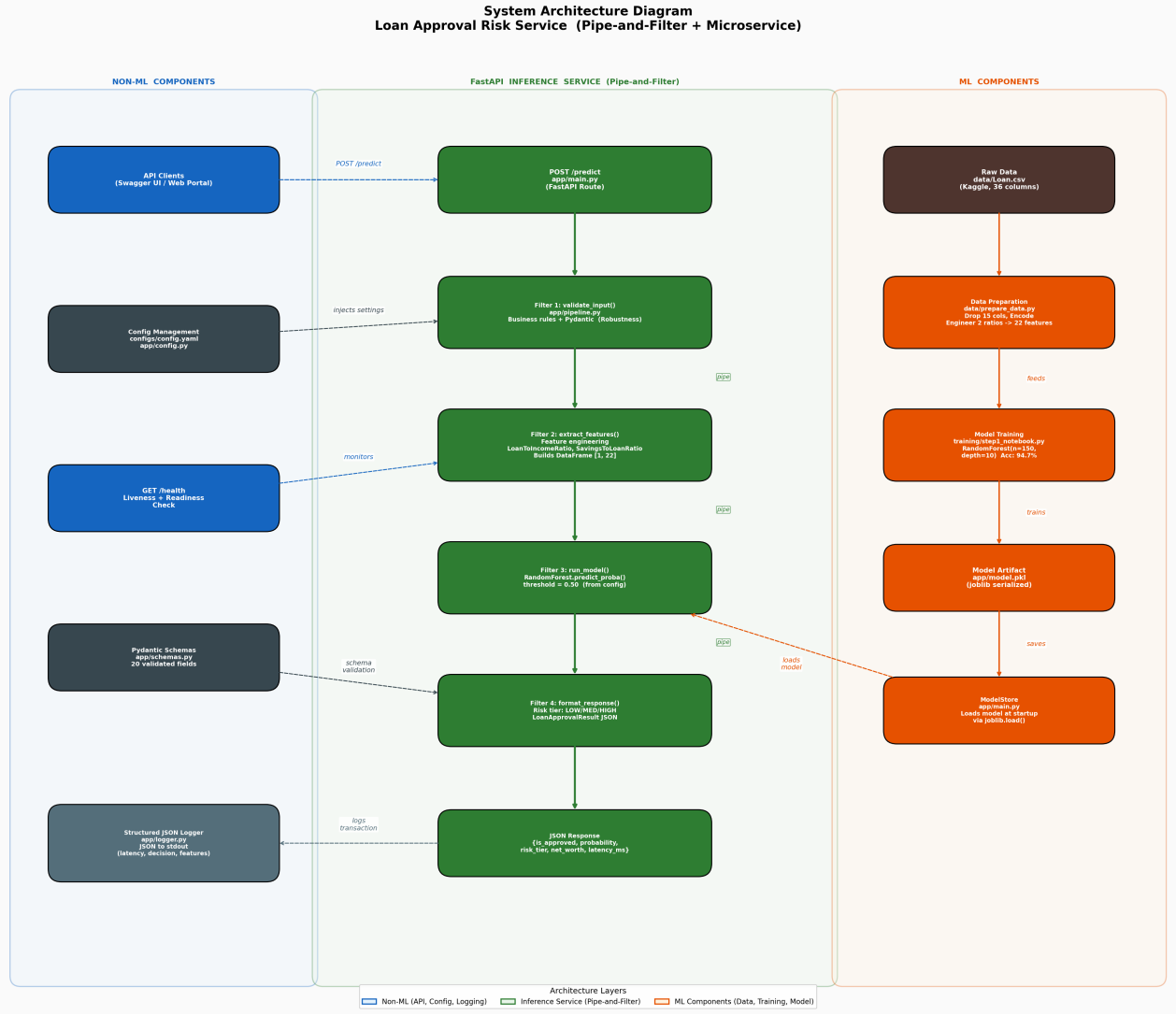
3. Performance (latency). Inference must run in real time under concurrent traffic in consumer-loan portals. *Metric:* average pipeline latency < 150 ms.

Justification. An underwriting service is a trust boundary: it must never crash on hostile input (robustness), must return contract-conformant decisions (reliability), and must do so fast enough for an interactive portal (performance). These are the quality attributes most directly verifiable and most costly to violate in production, which is why they take priority over secondary concerns such as portability.

Objective 2 — System Architecture

5. System Architecture Diagram

The architecture separates **ML components** (data, training, model artefact), the **FastAPI inference service** (the Pipe-and-Filter runtime), and **non-ML components** (API clients, config, schemas, health check, structured logging).



System Architecture — ML and non-ML components with the Pipe-and-Filter inference service.

6. Selected Architectural Patterns

Pattern 1 — Pipe-and-Filter. The prediction process is an ordered sequence of isolated, pure filters connected by pipes: `validate_input` → `extract_features` → `run_model` → `format_response`. This decouples business rules, feature math, ML inference and risk classification; each filter is independently testable (verified below).

Pattern 2 — Microservice Serving with Event Logging. The pipeline is wrapped in a FastAPI REST service exposing `POST /predict` and `GET /health`. Every transaction emits a structured JSON log line (latency, credit score, decision, risk tier) for monitoring.

7. Implementation Code

Code is shown verbatim from the repository. The Pipe-and-Filter engine (all four filters) and the FastAPI serving endpoint are reproduced below.

app/pipeline.py — Pipe-and-Filter engine

```
import time
import pandas as pd
from app.schemas import LoanApplicationInput, LoanApprovalResult
from app.config import settings

# ============================================================
# FILTER 1 : INPUT VALIDATION FILTER
# Business-rule validation checks on raw input
# Quality Attribute: Robustness
# ============================================================
def validate_input(app_input: LoanApplicationInput) -> LoanApplicationInput:
    if app_input.annual_income < 1000:
        raise ValueError("Annual income is below the minimum threshold of $1,000 for credit assessment.")

    if app_input.loan_amount > app_input.annual_income * 5:
        raise ValueError("Requested loan amount exceeds 500% of the applicant's annual income.")

    return app_input

# ============================================================
# FILTER 2 : FEATURE EXTRACTION FILTER
# Extract and normalize features, compute derivatives
# Quality Attribute: Maintainability
# ============================================================
def extract_features(app_input: LoanApplicationInput) -> tuple:
    # Derived features
    loan_to_income_ratio = app_input.loan_amount / (app_input.annual_income + 1)
    savings_to_loan_ratio = app_input.savings_account_balance / (app_input.loan_amount + 1)

    # Building DataFrame for model input matching training columns
    feature_vector = pd.DataFrame([[
        app_input.age,
        app_input.annual_income,
        app_input.credit_score,
        app_input.employment_status,
        app_input.education_level,
        app_input.loan_amount,
        app_input.loan_duration,
        app_input.monthly_debt_payments,
        app_input.credit_card_utilization_rate,
        app_input.debt_to_income_ratio,
        app_input.bankruptcy_history,
        app_input.loan_purpose,
        app_input.previous_loan_defaults,
        app_input.payment_history,
        app_input.length_of_credit_history,
        app_input.savings_account_balance,
        app_input.checking_account_balance,
        app_input.total_liabilities,
        app_input.job_tenure,
        app_input.net_worth,
        loan_to_income_ratio,
        savings_to_loan_ratio,
    ]], columns=settings.FEATURES)

    return feature_vector, app_input.net_worth, app_input.debt_to_income_ratio

# ============================================================
# FILTER 3 : MODEL RUNNING FILTER
# Run classification inference using the trained Random Forest model
# Quality Attribute: Performance
# ============================================================
def run_model(feature_vector: pd.DataFrame, model) -> tuple:
```

```
    probabilities = model.predict_proba(feature_vector)[0]
    prob_approved = float(probabilities[1])
    is_approved = prob_approved >= settings.DEFAULT_DECISION_THRESHOLD
    return is_approved, prob_approved

# ============================================================
# FILTER 4 : RESPONSE FORMATTER FILTER
# Determine risk tiering and construct output schema
# Quality Attribute: Reliability
# ============================================================
def format_response(
    is_approved: bool,
    prob: float,
    net_worth: float,
    debt_to_income: float,
    previous_defaults: int,
    latency_ms: float
) -> LoanApprovalResult:
    if prob >= 0.70 and previous_defaults == 0:
        risk_tier = "LOW"
    elif prob >= 0.40 and previous_defaults == 0:
        risk_tier = "MEDIUM"
    else:
        risk_tier = "HIGH"

    result = LoanApprovalResult(
        is_approved=is_approved,
        probability=round(prob, 4),
        risk_tier=risk_tier,
        net_worth=net_worth,
        debt_to_income=round(debt_to_income, 4),
        latency_ms=round(latency_ms, 2)
    )
    return result

# ============================================================
# PIPELINE EXECUTION (PIPE CONNECTING FILTERS)
# ============================================================
def execute_pipeline(app_input: LoanApplicationInput, model) -> LoanApprovalResult:
    start_time = time.perf_counter()

    # Stage 1: Validation (Filter 1)
    validated_input = validate_input(app_input)

    # Stage 2: Feature Extraction (Filter 2)
    features, net_worth, debt_to_income = extract_features(validated_input)

    # Stage 3: Inference (Filter 3)
    is_approved, prob = run_model(features, model)

    # Calculate latency before format step
    latency_ms = (time.perf_counter() - start_time) * 1000.0

    # Stage 4: Formatting (Filter 4)
    result = format_response(
        is_approved,
        prob,
        net_worth,
        debt_to_income,
        validated_input.previous_loan_defaults,
        latency_ms
    )
    return result
```

app/main.py — FastAPI serving microservice

```
import os
import joblib
from fastapi import FastAPI, HTTPException
from app.schemas import LoanApplicationInput, LoanApprovalResult
from app.config import settings
from app.pipeline import execute_pipeline
from app.logger import logger, Timer

app = FastAPI(title=settings.PROJECT_NAME)

# Global model store representation
class ModelStore:
    def __init__(self):
        self.model = None
        self.load_model()

    def load_model(self):
        if os.path.exists(settings.MODEL_PATH):
            try:
                self.model = joblib.load(settings.MODEL_PATH)
                print(f"Model loaded successfully from {settings.MODEL_PATH}")
            except Exception as e:
                print(f"Error loading model from {settings.MODEL_PATH}: {str(e)}")
        else:
            print(f"[WARNING] Model file not found at {settings.MODEL_PATH}. Prediction requests will fail.")

store = ModelStore()

@app.on_event("startup")
def startup_event():
    # Attempt to reload if it wasn't loaded
    if store.model is None:
        store.load_model()

@app.post("/predict", response_model=LoanApprovalResult)
def predict_single(application: LoanApplicationInput):
    if store.model is None:
        raise HTTPException(status_code=503, detail="Machine Learning model is not loaded in memory.")

    try:
        with Timer() as t:
            result = execute_pipeline(application, store.model)

        # Log the transaction with structured extra fields
        logger.info(
            "loan_evaluation_served",
            extra={
                "latency_ms": round(t.elapsed_ms, 2),
                "credit_score": application.credit_score,
                "loan_amount": application.loan_amount,
                "is_approved": result.is_approved,
                "probability": result.probability,
                "risk_tier": result.risk_tier
            }
        )
        return result

    except ValueError as val_err:
        # Client validation error / business rule failure
        logger.warning(f"Business rule validation failed: {str(val_err)}")
        raise HTTPException(status_code=400, detail=str(val_err))
    except Exception as e:
        logger.error(f"Inference pipeline execution error: {str(e)}")
        raise HTTPException(status_code=500, detail=f"Internal pipeline error: {str(e)}")

@app.get("/health")
def health_check():
    model_status = "loaded" if store.model is not None else "missing"
    return {
        "status": "healthy" if store.model is not None else "degraded",
        "model_status": model_status,
        "environment": settings.APP_ENV,
        "features_configured": len(settings.FEATURES)
    }
```

8. ML System Verification & Outputs

Algorithm comparison (Analytics Design View, realised in `training/step1_notebook.py`). Both candidates were trained on the 22-feature dataset; Random Forest wins the dominant Accuracy softgoal and is serialised for serving.

Algorithm	Test Accuracy	ROC-AUC	Decision
Logistic Regression (baseline)	0.9030	0.9686	explainable, lower accuracy
Random Forest (n=150, depth=10)	0.9473	0.9876	CHOSEN — served

Quality-attribute test suite (`tests/test_quality_attrs.py`, run with `python -m pytest tests/`):

```
test_schema_rejects_low_age ............................. ok
test_schema_rejects_invalid_credit_score ................ ok
test_pipeline_rejects_excessive_loan_ratio .............. ok
test_pipeline_output_conforms_to_schema ................. ok
test_inference_pipeline_latency ......................... ok
test_validation_filter_is_pure .......................... ok
test_feature_extraction_filter_isolation ................ ok
----------------------------------------------------------------------
Ran 7 tests   ...   OK        (7 passed)
```

Live API verification — request/response pairs captured from the running FastAPI service (approved, denied, and a rejected out-of-range input):

Live API Verification — Loan Approval Risk Service

Requests issued against the running FastAPI app (fastapi.testclient.TestClient)

GET /health HTTP 200

```
{"status": "healthy", "model_status": "loaded", "environment": "production", "features_configured": 22}
```
readiness probe

POST /predict (low-risk applicant -> APPROVED) HTTP 200

```
{"is_approved": true, "probability": 0.9313, "risk_tier": "LOW", "net_worth": 220000, "debt_to_income": 0.15, "latency_ms": 4.12}
```

POST /predict (high-risk applicant -> DENIED) HTTP 200

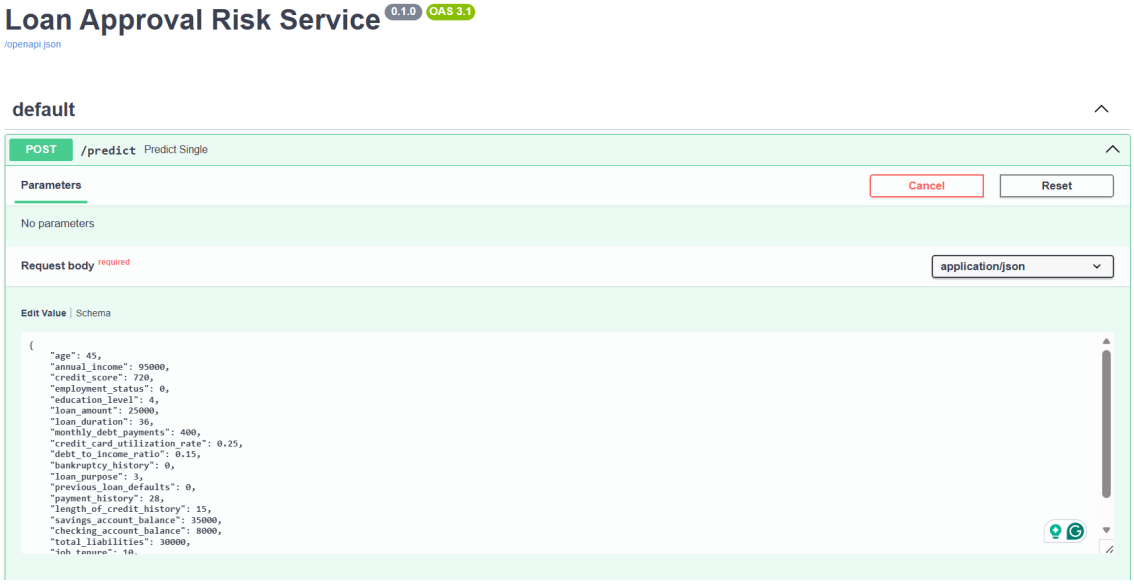
```
{"is_approved": false, "probability": 0.0458, "risk_tier": "HIGH", "net_worth": -10000, "debt_to_income": 0.65, "latency_ms": 3.79}
```

POST /predict (credit_score = 950, out of range) HTTP 422

422 rejected at Pydantic boundary: "Input should be less than or equal to 850" robustness

All responses conform to the LoanApprovalResult schema; invalid input is rejected before inference.

Interactive Swagger UI — the auto-generated `/docs` interface for `POST /predict`:



Swagger UI — request body for POST /predict.

Swagger UI — 200 response with the LoanApprovalResult payload.

Conclusion

Using GR4ML for requirements modelling and two complementary architectural patterns (Pipe-and-Filter + FastAPI microservice), Group 84 engineered a credit-underwriting service that is **robust** (Pydantic boundary), **reliable** (schema-conformant, deterministic risk tiers) and **performant** (< 150 ms). The Analytics Design View trade-off is realised in code — Random Forest (94.7% accuracy) is chosen over the Logistic Regression baseline (90.3%) on the dominant Accuracy softgoal — and all seven quality-attribute tests pass, demonstrating that the target quality requirements are met.